

Foundations — the math and proofs under every guard

The one track with homework.

The ESP32 and FPGA tracks show you **what** to build — an input turned into a guarded decision and a replayable trace. **Foundations** is the track that answers **why it's provably true**: it climbs, one rung at a time, from where an AS/BS in EE leaves off (Calc I/II, fixed-point numbers, a signal on a scope) all the way up to machine-checked certificates and the edge of what can be certified at all.

It's built the Monogate way — **hardware-anchored and honest**. Every rung starts from a real thing on the bench and pulls in exactly the math needed to answer a question about it. No "abstract first, hope you see why later."

How to read this track

Each rung is a lesson with the same five beats:

1. **The bench** — a real artifact (a scope trace, an evidence packet) and the question it raises.
2. **The one new idea** — the smallest new piece of math, in plain language.
3. **Do it by hand** — derive the bound yourself, on paper.
4. **Make it formal** — the matching Lean snippet (what the machine checks).
5. **Watch it hold** — the real hardware capture that confirms it.

You do **not** need to already know analysis, logic, or control theory. The first genuinely new skill isn't a math topic at all — it's *learning to read a proof*. We introduce that on Rung 1.5, and everything after leans on it.

You are here — the ladder

Rung	The bench question	The new idea	Anchored to
0	(you start here)	Calc I/II, fixed-point, a signal, some C	your EE background
1	How wrong is one hardware multiply?	absolute/relative error; truncation ≤ 1 LSB	<code>em1_sin.v</code> Q16.16
1.5	How do I know it's true for every input?	logic, $\forall/\exists$, direct proof & induction	<i>Book of Proof</i>
2	Where does the <code>x⁶</code> in the bound come from?	limits, the Mean Value Theorem, Taylor's theorem with remainder	the sin certificate's <code>x⁶</code>
3	Can I prove what the machine proved?	triangle inequality; error of <code>a×b</code> (composing bounds)	<code>sin_certificate_v0</code> , by hand
4	What does "sorryAx-free" actually mean?	propositions-as-types; Lean; Mathlib	<code>SinHardwareForwardError.lean</code>
5	Why can't the control loop ever escape?	state & dynamics; Lyapunov functions ; invariant sets	the <code>pid</code> closed-loop envelope
6	What functions can even be certified?	o-minimality; Pfaffian functions; Khovanskii fewnomials	the exponential-tower work

Rungs 1–3 are reachable with just your EE math plus a little proof practice. Rung 4 (Lean) and Rung 6 (the frontier) are real cliffs — the track says so honestly rather than pretending they're easy.

Reading & watching, per zone

Everything here is free or cheap, and gentle on purpose.

Proof literacy (Rung 1.5)

- *Book of Proof* — Richard Hammack (free PDF). The gentlest on-ramp to reading/writing proofs.
- *How to Prove It* — Daniel Velleman (if you want more practice).

Real analysis / Taylor (Rung 2)

- *Understanding Analysis* — Stephen Abbott. Famously the kindest first analysis book.
- *Introduction to Real Analysis* — William Trench (free PDF) as a reference.
- 3Blue1Brown, "Essence of Calculus" (YouTube) — for rebuilding intuition before the rigor.

Lean / formal methods (Rung 4)

- The **Natural Number Game** (browser, no install) — the perfect first taste of Lean.
- *Mathematics in Lean* and *Theorem Proving in Lean 4* (both free online) — the real references.

Monogate Foundations — the math & proofs under every guard : monogate.dev/electronics/foundations

- The Lean Zulip community — where you ask when stuck.

Control / dynamical systems (Rung 5)

- Steve Brunton's control lectures (YouTube) — EE-friendly and visual.
- *Feedback Systems* — Åström & Murray (free PDF). Lyapunov, stability, invariant sets.

The frontier (Rung 6)

- *Tame Topology and O-minimal Structures* — Lou van den Dries (the classic; hard — save it).
- Survey articles on Khovanskii's theorem / fewnomials for the ideas before the machinery.

Why this track exists

Formal verification and this flavor of applied analysis aren't taught at the AS/BS-EE level — they're barely mentioned. This is the curriculum built *while* learning it, anchored to real hardware we can measure, so every abstract claim ends with a scope trace. If you have Calc II and curiosity, you can start on Rung 1 today.

Lessons

The full ladder is written. Rungs 1–3 need only Calc II + a little proof practice; Rungs 4 and 6 are real cliffs (a new tool, and research-level math) and say so honestly.

The first arc — reachable now:

1. **Rung 1** — *How wrong is it, provably?* — fixed-point error; derive the `18/D` half of the certificate by hand.
2. **Rung 1.5** — *What even is a proof?* — the one skill an EE degree skips; $\forall/\exists$, direct proof, contradiction, induction.
3. **Rung 2** — *Where the `x6` comes from* — Taylor's theorem with remainder; derive the *whole* certificate.

The climb — the rest of the ladder: 4. **Rung 3** — *Composing error bounds* — triangle inequality

- `qm1_err`; why the composed `sin2+cos2` cert is looser than its parts.
5. **Rung 4** — *Make Lean check it (first cliff)* — propositions-as-types; what `sorryAx`-free guarantees; your first machine-checked proof.
 6. **Rung 5** — *Why the loop can't escape* — dynamics, invariant sets, Lyapunov functions; the closed-loop safety envelope, forever.
 7. **Rung 6** — *What can even be certified? (summit)* — o-minimality, Pfaffian functions, Khovanskii fewnomials; the research frontier.

Before you start: **Rung 0 — Orientation** — a 2-minute read on who the track is for, the 5-beat lesson format, and how to use the ladder. Each lesson also ends with a short **Exercises & checkpoint** so you can tell whether it stuck.

→ **New here?** Read **Rung 0**, then take **Rung 1**.